INVENTORY INTELLIGENCE DASHBOARD (IID)

A World-Class Decision Support System (DSS) integrating Stochastic Calculations, Dynamic Multi-Criteria Classifications, and ERP Readiness Gates.

Document Scope:	MPhil Thesis Reference & Operational Guide
Primary Author:	Lead Solutions Architect & Systems Engineer
Authentication Access:	Username: **admin** / Password: **admin123**
Database Engine:	SQLite (Local) / PostgreSQL Ready
Date of Release:	2026-08-24

1. System Abstract & Design Architecture

The Inventory Intelligence Dashboard (IID) is designed as a night-operations command center for modern supply chains. It acts as a decision support framework, transforming raw warehouse datasets into actionable operational instructions. Built on a decoupled SPA (Single Page Application) frontend layout utilizing HTML, CSS, and vanilla JavaScript, it connects to a lightweight and high-throughput Python API served by FastAPI and backed by SQLAlchemy ORM.

For deployment ease and thesis demonstration, IID mounts the frontend files directly onto the FastAPI server process, allowing the entire platform to start with a single console command. The system automatically initializes and structures a local SQLite database (`iid_local.db`) if not present, seeding it with the initial user access credentials.

2. User Access & Security Protocols

User operations are secured behind a custom authentication gateway in `backend/api/auth.py`. To eliminate heavy framework dependencies and compile-time issues on target environments, password security is implemented via high-strength SHA-256 hashing. Passwords are salted with a standard system salt before hashing.

Seeding Default Credentials: On server startup, the database is audited and automatically seeds a default user if the table is empty.

Default Credentials:

• Username: **admin**
• Password: **admin123**

How to Modify Credentials Programmatically:

To change the pre-seeded admin credentials before building, navigate to `backend/api/auth.py` and modify the `seed_admin_user` function:

```
def seed_admin_user(db: Session):
    admin = db.query(User).filter(User.username == "NEW_USERNAME").first()
    if not admin:
        admin = User(
            username="NEW_USERNAME",
            password_hash=hash_password("NEW_PASSWORD"),
            role="admin"
        )
        db.add(admin)
        db.commit()
```

Note: If you have already run the application once, delete the SQLite file `backend/database/iid_local.db` before restarting. The system will recreate the database and seed it with your newly specified credentials automatically.

3. Optimization & Academic Formulation Models

The mathematical optimization engine lies in `backend/optimization/stock_optimizer.py` and `backend/analytics/analytics_engine.py`. It supports advanced academic and doctoral research inventory optimization formulations:

Economic Order Quantity (EOQ)

$$EOQ = \sqrt{\frac{2 \cdot D \cdot S}{H}}$$
Where:
• **D**: Annual demand in units (Daily Average Demand × 365)
• **S**: Fixed order setup cost (configured at a standard $50.00)
• **H**: Holding cost per unit per year (calculated as unit cost × 20% annual holding rate)

Stochastic Safety Stock (SS) under Variance

$$SS = Z \times \sqrt{L \cdot \sigma_D^2 + D^2 \cdot \sigma_{LT}^2}$$
Where:
• **Z**: Standard service factor (fixed at 1.65 for a 95% service fill rate target)
• **L**: Vendor lead time in days
• **D**: Daily average demand in units
• **sigma_D**: Standard deviation of demand (stochastically calculated at 25% of daily demand)
• **sigma_LT**: Standard deviation of lead time (stochastically calculated at 15% of lead time)

Stochastic Reorder Point (ROP)

$$ROP = (D \times L) + SS$$
Triggers replenishment orders whenever the quantity on hand falls below the ROP threshold.

Multi-Criteria Decision Classifications

• **ABC Analysis**: Classifies inventory by annual cost usage: **A** (top 80% usage), **B** (next 15%), **C** (remaining 5%).
• **XYZ Analysis**: Segmented by demand predictability (Coefficient of Variation): **X** (CV ≤ 10%), **Y** (CV ≤ 30%), **Z** (CV > 30%).
• **FSN Analysis**: Classifies inventory speed based on turnover ratio: **Fast (F)**, **Slow (S)**, **Non-moving (N)**.
• **VED Analysis**: Segments inventory by operational criticality buffer: **Vital (V)**, **Essential (E)**, **Desirable (D)**.

4. Editable and Touchable Telemetry

Every single table element rendered across the pages (including Overstock, Expiry, Safety Stock, Warehouses, and Suppliers) is wired with the `touchable-row` class. Clicking any row opens a Control Tower details modal.

From this control tower, users can edit parameters in real time. Clicking **Save Changes** submits a `PUT` request to `/api/update/*`, committing the modifications to the database, refreshing the active view, and re-calculating the global KPIs. Clicking **Delete Entity** prompts a double confirmation check, executing a `DELETE` request, clearing the warehouse cache, and refreshing the active view.

5. Step-by-Step Portability & Deployment Guide

To run this application on any other laptop, follow this guide precisely. No build compilation or heavy tools are required.

Step A: Prepare Python Environment

1. Download and install **Python 3.10+** (ensure you check 'Add Python to PATH' during installation).
2. Copy the entire project folder (`Inventory-Intelligence-Dashboard`) onto the new laptop.

Step B: Configure the Local Virtual Environment

1. Open Command Prompt (cmd) or PowerShell on the laptop.
2. Navigate into the `backend/` directory:
   cd D:\path\to\Inventory-Intelligence-Dashboard\backend
3. Create a clean virtual environment:
   python -m venv venv
4. Activate the virtual environment:
   Windows cmd: **.\venv\Scripts\activate**
   PowerShell: **.\venv\Scripts\activate.ps1**
   macOS/Linux: **source venv/bin/activate**
5. Install all libraries locally from PyPI:
   pip install -r requirements.txt

Step C: Start the Dashboard

1. Run the FastAPI server in the active console:
   uvicorn main:app --reload --port 8000
2. Launch any modern browser and open:
   http://127.0.0.1:8000
3. Enter username: **admin** and password: **admin123** to log in and build your control tower.